

第12章实验 并发控制

并发控制有1个实验项目（参见表11），该实验项目属于选修的设计性实验。

表11 “并发控制”实验项目一览表

序号	实验项目名称	开设类别	难易程度	建议实验学时	对应教材章节
实验 12.1	并发控制	选修	适中	2	第12章

1. 实验内容和要求

掌握数据库并发控制的基本原理及其应用方法。验证并发操作带来的数据不一致性问题，包括丢失修改、脏读、不可重复读、幻读等多种情况。要求通过取消查询分析器的自动提交功能，创建两个不同的用户，分别登录查询分析器，同时打开两个客户端；通过使用 SQL 语句设计具体示例，展示不同封锁级别的应用场景，验证各种封锁级别的并发控制效果，以进一步理解封锁技术如何解决事务并发导致的问题。

实验重点：并发操作带来的数据不一致性问题。

实验难点：设计具体的示例以演示各种封锁级别。

2. 实验报告示例

实验报告			
题目：并发控制实验	姓名	日期	
DBMS 通常提供 4 种隔离级别：读未提交(read uncommitted)、读已提交(read committed)、可重复读(repeatable read)和可串行化(serializable)。 4 种隔离级别与三级封锁协议的大致对应关系如下： ① 读未提交隔离级别提供最大的事务并发度，但仅能避免丢失修改，对应一级封锁协议；读已提交隔离级别提供的事务并发度减弱，能够避免丢失修改和脏读，对应二级封锁协议。 ② 可重复读隔离级别提供的事务并发度进一步减弱，能够避免丢失修改、脏读和不可重复读。 ③ 可串行化隔离级别提供最小的事务并发度，能够避免所有的事务并发控制问题，对应三级封锁协议。 read committed 是 KingbaseES 默认的事务隔离级别。对于运行在这种隔离级别的事务，查询语句只能看到该查询开始执行之前提交的数据，而不会看到任何未提交的数据或查询执行期间并发的其他事务提交的数据，但是该查询可以看到本事务中查询之前执行的数据更新。			

续表

serializable 提供了更加严格的事务隔离级别。在这种事务隔离级别下,事务好像没有并发,是串行执行的。对于运行在 serializable 隔离级别的事务,查询语句只能看到该事务开始执行之前提交的数据,而不会看到任何未提交的数据或事务执行期间并发的其他事务提交的数据。

repeatable read 向上兼容 serializable。read uncommitted 向上兼容 read committed。

1. 实验准备

(1) 数据准备

给 Sales 模式下的 PartSupp 表增加一条记录。

```
/* 插入零件供应记录 */
INSERT INTO Sales.PartSupp(Partkey, Suppkey, Availqty, Supplycost, Comment)
VALUES(1, 1, 0, 0.0, null);
/* 设置指定零件供应记录的可用数量为 0 */
UPDATE Sales.PartSupp
SET Availqty = 0
WHERE Partkey = 1 AND Suppkey = 1;
```

(2) 创建两个超级用户 SYSTEM1 和 SYSTEM2

```
CREATE USER SYSTEM1 WITH SUPERUSER PASSWORD 'MANAGER';
CREATE USER SYSTEM2 WITH SUPERUSER PASSWORD 'MANAGER';
```

(3) 分别以 SYSTEM1 和 SYSTEM2 用户登录查询分析器

假设 SYSTEM1 用户登录打开的查询分析器称为客户端 A, SYSTEM2 用户登录打开的查询分析器称为客户端 B。设置客户端 A 和客户端 B 的当前数据库为 TPC-H。取消两个查询分析器的自动提交事务的功能,改为显式事务提交,即需要执行 COMMIT 命令提交事务。

```
SHOW TRANSACTION ISOLATION LEVEL;
```

2. 验证丢失情况

在“读已提交”隔离级别下,验证丢失修改的情况。

① 查看 Sales 模式下 PartSupp 表的 Availqty 值为 0。

分别在客户端 A 和客户端 B 执行如下 SQL 语句:

```
/* 查看零件供应记录 */
SELECT Availqty FROM Sales.PartSupp WHERE Partkey = 1 AND Suppkey = 1;
```

② 在客户端 A 执行如下 SQL 语句,修改 PartSupp 表的 Availqty 值:

```
UPDATE Sales.PartSupp SET Availqty = Availqty - 5
WHERE Partkey = 1 AND Suppkey = 1;
```

续表

/*查看 PartSupp 表的 Availqty 值为修改后的值*/

SELECT Availqty FROM Sales.PartSupp WHERE Partkey =1 AND Suppkey = 1;

- ③ 在客户端 B 执行如下 SQL 语句，也修改 PartSupp 表的 Availqty 值：

UPDATE Sales.PartSupp SET Availqty =Availqty - 5

WHERE Partkey = 1 AND Suppkey = 1;

/*查看 PartSupp 表的 Availqty 值为修改后的值*/

SELECT Availqty FROM Sales.PartSupp WHERE Partkey =1 AND Suppkey = 1;

- ④ 在客户端 A 执行如下 SQL 语句提交事务：

COMMIT;

- ⑤ 在客户端 B 执行如下 SQL 语句提交事务：

COMMIT;

- ⑥ 在客户端 A 和 B 分别执行如下 SQL 语句，查看 PartSupp 表的 Availqty 值：

/*看到修改后且已提交的 Availqty 值*/

SELECT * FROM Sales.PartSupp WHERE Partkey =1 AND Suppkey = 1;

上述操作过程可以示意如下：

T _A 终端 A 提交的事务	T _B 终端 B 提交的事务
Read(Availqty)=22	Read(Availqty)=22
Update(Availqty)=Availqty-5	Update(Availqty) = Availqty-5
Read(Availqty)=17	等待...
Commit	Update(Availqty) = Availqty-5 执行完成
	Read(Availqty)=12
	Commit;

续表

3. 验证避免脏读的情况

在“读已提交”隔离级别下，验证避免脏读的情况：

① 查看 Sales 模式下 PartSupp 表的 Availqty 值为 0。

分别在客户端 A 和客户端 B 执行如下 SQL 语句：

```
/* 查看零件供应记录 */
```

```
SELECT Availqty FROM Sales.PartSupp WHERE Partkey =1 AND Suppkey = 1;
```

② 在客户端 A 执行如下 SQL 语句，以修改 PartSupp 表的 Availqty 值：

```
UPDATE Sales.PartSupp SET Availqty =22 WHERE Partkey = 1 AND Suppkey = 1;
```

```
/*查看 PartSupp 表的 Availqty 值为修改后的值*/
```

```
SELECT Availqty FROM Sales.PartSupp WHERE Partkey =1 AND Suppkey = 1;
```

③ 在客户端 B 执行如下 SQL 语句，查看 PartSupp 表的 Availqty 值：

```
/*看不到 A 客户端修改后但尚未提交的 Availqty 值*/
```

```
SELECT Availqty FROM Sales.PartSupp WHERE Partkey =1 AND Suppkey = 1;
```

④ 在客户端 A 执行如下 SQL 语句提交事务：

```
COMMIT;
```

⑤ 在客户端 B 执行如下 SQL 语句，查看 PartSupp 表的 Availqty 值：

```
/*看到 A 客户端修改后且已提交的 Availqty 值*/
```

```
SELECT * FROM Sales.PartSupp WHERE Partkey =1 AND Suppkey = 1;
```

上述操作过程可以示意如下：

T _A 终端 A 提交的事务	T _B 终端 B 提交的事务
Read(Availqty)=0	
	Read(Availqty)=0
Update(Availqty)=22	
Read(Availqty)=22	
	Read(Availqty)=0
Commit	
	Read(Availqty)=22

4. 验证出现不可重复读的现象

在“读已提交”隔离级别下，验证出现不可重复读的现象。

续表

- ① 客户端 A 执行如下 SQL 语句, 读取指定零件的 Availqty 值:

```
SELECT Availqty FROM Sales.PartSupp WHERE Partkey =1 AND Suppkey = 1;
```

- ② 客户端 B 执行如下 SQL 语句, 修改指定零件的 Availqty 值:

```
UPDATE Sales.PartSupp SET Availqty =33 WHERE Partkey = 1 AND Suppkey = 1;
```

- ③ 客户端 A 再次读取指定零件的 Availqty 值:

```
/*发现这次读取的值与上次读取的值一样, 因为客户端 B 还没有提交*/
```

```
SELECT Availqty FROM Sales.PartSupp WHERE Partkey =1 AND Suppkey = 1;
```

- ④ 客户端 B 提交事务:

```
COMMIT;
```

- ⑤ 客户端 A 再次读取指定零件的 Availqty 值:

```
/*发现这次读取的值与上次读取的值不一样了, 发生了不可重复读现象*/
```

```
SELECT Availqty FROM Sales.PartSupp WHERE Partkey =1 AND Suppkey = 1;
```

- ⑥ 客户端 A 提交事务:

```
COMMIT;
```

5. 在“读已提交”隔离级别下验证出现“幻影”的现象

- ① 客户端 A 执行如下 SQL 语句, 读取指定供应商的零件供应记录。

```
SELECT * FROM Sales.PartSupp WHERE Suppkey = 1;
```

- ② 客户端 B 执行 SQL 语句, 插入指定供应商新的零件供应记录。

```
INSERT INTO Sales.PartSupp(Partkey, Suppkey, Availqty, Supplycost, Comment)  
VALUES(2, 1, 0, 0.0, null);
```

- ③ 客户端 A 再次读取指定供应商的零件供应记录。

```
/*发现这次读取的值与上次读取的值一样, 因为客户端 B 还没有提交*/
```

```
SELECT * FROM Sales.PartSupp WHERE Suppkey = 1;
```

- ④ 客户端 B 提交事务。

```
COMMIT;
```

- ⑤ 客户端 A 再次读取指定供应商的零件供应记录。

续表

```
/*发现这次读取的值与上次读取的值不一样了，发生了“幻影”现象*/  
SELECT * FROM Sales.PartSupp WHERE Suppkey = 1;
```

⑥ 客户端 A 提交事务。

```
COMMIT;
```

6. 在“可串行化”隔离级别下验证出现“幻影”的现象

修改 KingbaseES 的 data 文件下的 kingbase.conf 文件，设置 default_transaction_isolation 为 'serializable'，重新启动数据库，然后重新执行本实验中 2、3 和 4 三项实验。验证在“可串行化”隔离级别下，避免脏读、避免不可重复读和避免出现“幻影”的情况。

实验总结：

不可重复读分为三种情况：一是由于修改而导致不可重复读；二是由于插入新记录而导致不可重复读，第二次读取时会多出一些新记录；三是由于删除记录而导致不可重复读，即第二次读取时有些旧记录“消失”了。后面两种情况称为“幻影”现象。

3. 思考题

- ① 试分析在“读已提交”隔离级别下，设计一个应用程序以避免“不可重复读”和“幻影”现象。
- ② 对比分析不同数据库实现隔离级别的异同点。

五、实验考核标准和评价方式

1. 实验考核标准

数据库课程实验考核内容主要分为三部分：实验过程考核、实验代码考核和实验报告考核。实验过程考核主要考查学生的实验课程出勤、实验态度、实验任务完成情况、实验过程中分析问题和解决问题的能力等方面。实验代码考核主要是考查学生编写的 SQL 代码的规范性、正确性和代码质量等方面。实验报告考核主要是考查实验报告内容的完整性、正确性和规范性等方面（实验考核评分标准表见表 12 所示）。

数据库课程实验通常由教师选择实验手册中的若干个小实验组成，实验成绩通常占课程总成绩的 30%~50%，实验过程、实验代码和实验报告可分别占三分之一的分数。考核标准以实验成绩 100 分计算，最终实验成绩可以根据其占课程总成绩的百分比折算。

表 12 实验考核评分标准表

考核内容	评分项	评分标准	分数	备注
1. 实验过程	1.1 实验课程出勤	全勤	10	
		缺勤 2 次以下	8	
		缺勤 3 次以上	6	
	1.2 实验态度	积极认真	10	
一般		7		
态度不端正		5		
1.3 实验任务完成情况	全部完成	10		
	缺项 1 次	8		
	缺项 2 次以上	6		
1.4 分析问题和解决问题的能力	良好	10		
	一般	7		
	较差	6		
2. 实验代码	2.1 代码规范性	符合 SQL 编程规范	10	
		基本规范	8	
		不够规范	6	
	2.2 代码正确性	运行结果全部正确	10	
错误 2 项以下		7		
错误 3 项以上		5		
2.3 代码质量	质量较高	10		